

Backend / Tooling Task

This task is a chance to show how you take a loosely defined problem and turn it into a working tool. We care as much about how you think and structure the solution as about the final output.

The problem

You will receive a CSV containing a list of product IDs from Myntra.

Build a tool that takes this list and, for each product, fetches a set of details from the public Myntra product page and returns them in a structured, usable form.

Input

A CSV file (provided separately) containing:

- `product_id`: the Myntra product identifier

Core requirements

For each product ID, fetch and return:

- Title
- Description
- One or two product images
- Rating
- Total ratings count
- Category

Category ad results: for the product's category, return the first 3 Ad-marked (sponsored) results only from the search page for that category, each with its rating and price. Nothing beyond the first 3.

Your tool should be robust. It should handle errors and missing or unavailable data without crashing, and clearly report anything it could not process.

Output should be structured and machine-readable (for example, JSON), in addition to however you choose to display it. Frontend hosted tool will be a preferable option.

Bonus (optional, not required)

These are stretch goals. Do not sacrifice the quality of the core for them.

- Delivery check: for a few pincodes from Bengaluru, Mumbai, Delhi, Ahmedabad, and Kolkata, fetch the estimated delivery days for the product.
- Frontend: a simple, clean interface where the list can be submitted and the results viewed (for example, a results table).

Scope and approach

Treat the time budget as roughly two days.

Part of what we are looking for is how you scope. If you think some parts are larger than two days, tell us. We would rather see a solid, reliable core with clear notes on what you would build next than a fragile attempt at all of it.

Use whatever tools, libraries, and approach you are comfortable with. Python is preferred for the backend.

A few things to keep in mind

- Use only publicly accessible product pages. No login credentials are involved.
- Be mindful of how marketplaces respond to automated requests. How you handle that is part of the exercise.
- Missing data is normal. Decide how your tool should behave when a field is not available.

What to submit

1. The code, in a Git repository (or a zip if you prefer).
2. A short README covering:
 - How to run it.
 - The approach you took, and why.
 - Any assumptions you made.
 - What you scoped in versus out, and what you would build next with more time. (**Important**)
 - Anything that does not work, or that you would improve.
3. A sample of the output produced from the provided list.

How we will evaluate

- Whether the core works reliably, including on missing or unavailable data and other edge cases.
- How you handle the parts that do not go smoothly (blocks, errors, edge cases).
- Code clarity and structure.
- The quality of your scoping and the reasoning in your README.
- Communication. We would rather get a question from you than a wrong assumption built out in full.

If anything is unclear, ask. Knowing when to ask is part of the job.